

Tema 5 — Búsqueda Informada

Guía de estudio · Razonamiento y Planificación Automática · UNIR

Cómo usar esta guía

Esta guía está organizada en cuatro bloques que cubren los algoritmos informados del Tema 5. Cada bloque combina la intuición inicial, la formalización matemática, ejemplos numéricos resueltos, y las dudas frecuentes que surgieron en la sesión de estudio. Al final incluye el test de autoevaluación con respuestas comentadas.

1. Heurísticas: los cimientos

1.1 ¿Qué es una heurística?

La palabra viene del griego *heuriskein* = "encontrar, descubrir". En inteligencia artificial representa el **conocimiento extraído de la experiencia dentro del dominio del problema**, usado para estimar qué tan prometedor es un estado sin resolver el problema completo.

Analogía central

Cuando jugás ajedrez, no calculás el resultado final de todas las jugadas posibles. Tu cerebro *estima* con la experiencia cuál se ve más prometedora. Esa estimación es una heurística.

La materialización técnica es la **función heurística**:

Función heurística $h(n)$

Función dependiente del estado que estima el coste desde el nodo n hasta la meta más cercana.

$$h(n) : n \rightarrow G \quad (\text{coste estimado})$$

$$h^*(n) : n \rightarrow G \quad (\text{coste real, generalmente desconocido})$$

1.2 Las dos funciones clave

En búsqueda informada trabajamos con dos magnitudes que hay que diferenciar muy bien:

Función	Qué mide	Desde	Hasta	Naturaleza
$g(n)$	coste	inicio	nodo n	real (ya recorrido)
$h(n)$	coste	nodo n	meta	estimado

Duda de la sesión: nodos vs piezas en puzzle-8

Confusión frecuente: en puzzle-8, un **nodo**

es una configuración completa del tablero 3x3 (una "foto" del tablero), no una pieza individual. Una **pieza** es una ficha dentro de esa configuración. Los vecinos de un nodo son los tableros que resultan de mover la pieza vacía una casilla.

1.3 Heurísticas admisibles (optimistas)

Heurística admisible

Una función heurística h es admisible si para todo nodo n :

$$h(n) \leq h^*(n)$$

Es decir,

nunca sobreestima

el coste real hasta la meta. También se llama *optimista*.

Ejemplos de heurísticas admisibles

- **Mapas de carreteras:**

distancia en línea recta (euclídea). Siempre $\leq$ distancia por carretera.

- **Puzzle-8 — piezas descolocadas:**

número de fichas fuera de su posición final. Siempre $\leq$ movimientos reales necesarios.

- **Puzzle-8 — distancia Manhattan:**

suma de distancias de cada ficha a su posición final, medida en movimientos ortogonales. Admisible y más informada que piezas descolocadas.

- **Heurística trivial:**

$h(n) = 0$ para todo n . Admisible pero inútil (no da información).

¿Por qué importa la admisibilidad?

Si la heurística *sobreestima* el coste, el algoritmo A* puede descartar el camino óptimo confundido por valores inflados artificialmente. Esto rompe la garantía de optimalidad. Por eso necesitamos heurísticas que "pequen por lo bajo" — siempre subestimar o acertar, nunca inflar.

1.4 Heurísticas consistentes (monótonas)

Heurística consistente

Una heurística h es consistente si para cualquier nodo s y cualquier sucesor

s' de s :

$$h(s) \leq c(s, s') + h(s')$$

donde $c(s, s')$ es el coste real del arco que va de s a s' .

Analogía: el GPS coherente

Una heurística consistente es como un GPS que nunca se contradice. Si dice "faltan 50 km" y avanzaste 10 km, ahora no puede decir "faltan 80 km". Puede decir 40, 45, incluso 50 (si el tráfico empeoró), pero nunca puede inventar kilómetros que antes no existían.

Propiedad estrella: f es monótonamente creciente

Si h es consistente, entonces $f(n) = g(n) + h(n)$ nunca decrece al avanzar por

un camino. Es decir, para cualquier sucesor s' de s :

$$f(s) \leq f(s')$$

Demostración breve

Partimos de la definición de consistencia:

$$h(s) \leq c(s, s') + h(s')$$

Sumamos $g(s)$ a ambos lados:

$$g(s) + h(s) \leq g(s) + c(s, s') + h(s')$$

Pero $g(s) + c(s, s') = g(s')$ (el coste acumulado del padre más el arco es el del hijo).

Entonces:

$$g(s) + h(s) \leq g(s') + h(s')$$

$$f(s) \leq f(s') \quad \blacksquare$$

Relación admisibilidad $\leftrightarrow$ consistencia

Toda heurística consistente es admisible, pero no al revés

Consistentes $\subset$ Admisibles

Por eso diseñar una heurística consistente es "dos pájaros de un tiro": ganas admisibilidad automáticamente.

1.5 Calidad de heurísticas

Más informada

Si h_1 y h_2 son ambas admisibles, decimos que h_1 es más informada que h_2 si para todo nodo n :

$$h_1(n) \geq h_2(n)$$

Una heurística más informada da valores más cercanos al coste real (sin pasarse). Cuanto más informada, menos nodos expande A*. Combinación útil de múltiples heurísticas admisibles:

$$h(n) = \max(h_1(n), h_2(n), \dots, h_m(n))$$

El máximo de varias admisibles sigue siendo admisible, y es al menos tan informada como la mejor de ellas.

Duda de la sesión: ¿el ejemplo Madrid-Zaragoza original era consistente?

En el primer ejemplo ($h(\text{Madrid}) = 500$, $c = 300$, $h(\text{Zaragoza}) = 300$), la heurística **sí es consistente**, aunque parezca "contradictoria" a primera vista.

Verificación: $500 \leq 300 + 300 = 600$ ✓

Lo que pasa es que f pasa de 500 a 600 al avanzar — sube. Eso está permitido (monotonía creciente). La consistencia NO exige que la estimación total se mantenga fija; exige que **no decrezca**. El caso inconsistente sería $h(\text{Madrid}) = 700$ con

$700 > 600$, donde f bajaría al avanzar — eso es lo prohibido.

2. Algoritmo A*

2.1 La intuición

UCS (Tema 4) ordena por $g(n)$ — solo mira hacia atrás (lo ya gastado). A* agrega mirada hacia adelante con la heurística, combinando ambas informaciones:

Función de evaluación de A*

$$f(n) = g(n) + h(n)$$

- $g(n)$ = coste real acumulado desde el inicio hasta n
- $h(n)$ = coste estimado desde n hasta la meta
- $f(n)$ = coste total estimado del camino que pasa por n

A* siempre expande el nodo de la cola abierta con el

menor $f(n)$

.

Analogía: taxi miope vs estratégico

UCS es como tomar el taxi que te deja en la esquina más cercana barata (mira solo el costo actual). A* es como elegir el taxi que te deja cerca del destino final aunque cobre un poco más (mira el viaje completo estimado).

2.2 Pseudocódigo

1. Inicializar colaAbierta con el nodo inicial S_0
2. $S_0.g \leftarrow 0$
3. $S_0.f \leftarrow h(S_0)$ // porque $g(S_0) = 0$
4. Mientras colaAbierta no esté vacía:

```

5.     nodo ← extraer el de MENOR f de colaAbierta
6.     Si nodo es meta → retornar camino
7.     sucesores ← expandir(nodo)
8.     Para cada sucesor:
9.         g_tentativo ← nodo.g + c(nodo, sucesor)
10.        Si g_tentativo < sucesor.g (es mejor camino):
11.            sucesor.padre ← nodo
12.            sucesor.g ← g_tentativo
13.            sucesor.f ← sucesor.g + h(sucesor)
14.            Agregar sucesor a colaAbierta
15. Retornar "sin solución"

```

Criterio crítico de terminación

A* termina cuando la meta es *extraída* de la cola como nodo de menor f , NO cuando es agregada. Parar antes puede retornar una solución subóptima.

2.3 Ejemplo numérico completo: mapa rumano

Objetivo: llegar a B (Bucarest) desde A (Arad). Heurística: distancia en línea recta.

Heurística h a Bucarest:

A	S	T	Z	R	F	P	B
366	253	329	374	193	178	98	0

Corrida paso a paso

Paso 1 — Inicio en A

$$g(A) = 0 \quad , \quad h(A) = 366 \quad , \quad f(A) = 366 \quad . \text{ Cola: } \{A:366\}$$

Paso 2 — Expandir A

(sucesores S, T, Z)

Nodo	g	h	$f = g + h$
S	140	253	393
T	118	329	447
Z	75	374	449

Observación: Z tiene el menor g (75), pero A* elige S porque tiene el menor f (393). La heurística indica que S está más cerca de Bucarest.

Paso 3 — Expandir S

(sucesores R, F)

Nodo	g	h	f
R	220	193	413
F	239	178	417

Paso 4 — Expandir R

(sucesor P)

$$g(P) = 317 \quad , \quad h(P) = 98 \quad , \quad f(P) = 415$$

Paso 5 — Expandir P

(sucesor B)

$$g(B) = 418 \quad , \quad h(B) = 0 \quad , \quad f(B) = 418$$

B entra a la cola con $f = 418$, pero F tiene $f = 417$. NO paramos.

Paso 6 — Expandir F

(sucesor B por otro camino)

$$g(B) = 450 \quad \text{— peor que 418, se descarta.}$$

Paso 7 — Extraer B

(ahora es el menor f de la cola)

¡Meta alcanzada! Camino óptimo: $A \rightarrow S \rightarrow R \rightarrow P \rightarrow B$, coste total = 418

.

2.4 Propiedades de A*

Propiedad	¿Se cumple?	Condición
Compleitud	Sí	Existe solución
Optimalidad (árbol)	Sí	h admisible
Optimalidad (grafo con cerrados)	Sí	h consistente
Complejidad temporal	$O(b^d)$ en peor caso	Depende de calidad de h
Complejidad espacial	Alta	Guarda todos los nodos abiertos

Casos extremos de la heurística

- $h(n) = 0$ para todo n : A* degenera en UCS (pura mirada atrás).
- $h(n) = h^*(n)$ (perfecta): complejidad lineal, cero nodos "desperdiciados".
- Entre medio: cuanto más informada sea h sin perder admisibilidad, menos nodos expande.

Comparación numérica 8-puzzle

Profundidad d	No informada	A* piezas descolocadas	A* Manhattan
4	112	13	12
8	6.384	39	25
12	3.644.035	227	73
20	—	7.276	676
24	—	39.135	1.641

Manhattan es más informada que piezas descolocadas → expande muchos menos nodos.

Duda de la sesión: ¿qué se pierde con heurística inadmisibile?

Se pierde la
optimalidad

, pero **NO la completitud**. A* con h inadmisibile sigue encontrando alguna solución (si existe), pero no garantiza que sea la mejor. La completitud depende del mecanismo de búsqueda (cola de prioridad + exploración sistemática), no de la calidad de h .

3. Búsqueda por subobjetivos

3.1 Aplicación débil vs fuerte de heurísticas

Tipo	Rol de h	Ejemplo	Garantías
Débil	Guía la búsqueda, ordena prioridades	A*	Mantiene completitud y optimalidad
Fuerte	Reduce el espacio de búsqueda	Subobjetivos, ascenso colinas	Puede perder optimalidad y/o completitud

3.2 La idea

En lugar de buscar un plan completo de s_0 a s_m , identificamos una secuencia de estados intermedios $i_1, i_2, \dots, i_n$ por los que probablemente pasa el camino óptimo, y resolvemos $n + 1$ subproblemas pequeños:

- De s_0 a i_1
- De i_1 a i_2
- ...
- De i_n a s_m

Cada subproblema se resuelve con un método base (BFS, DFS, A*, el que sea). Las soluciones se concatenan.

Analogía: viaje Manizales → Bogotá

En lugar de dejar al GPS explorar todas las rutas posibles, le imponés: "pasá sí o sí por Honda y por Villeta". Resuelve tres problemitas (Manizales→Honda, Honda→Villeta, Villeta→Bogotá) en vez de uno gigante. Mucho más rápido, pero si la mejor ruta no pasaba por Honda, te quedás con una subóptima.

3.3 Complejidad: la gran ganancia

Fórmulas de complejidad (peor caso, tiempo)

$$\text{Sin subobjetivos: } \frac{b^{d+1} - 1}{b - 1} - 1$$

$$\text{Con subobjetivos: } (x + 1) \cdot \frac{b^{d'+1} - 1}{b - 1} - 1$$

donde b es factor de ramificación, d profundidad total, x número de subobjetivos, d' profundidad de cada subproblema.

Ejemplo numérico

Caso: $b = 3$, $d = 20$, $x = 4$ subobjetivos de profundidad $d' = 5$.

	Búsqueda directa	Por subobjetivos
Mejor caso (tiempo)	1.743.392.200	605
Mejor caso (espacio)	5.230.176.601	1.820
Peor caso (tiempo)	5.230.176.600	1.815
Peor caso (espacio)	15.690.529.801	5.450

Cálculo de ratio de magnitud con logaritmos

Para $b = 5$, $d = 30$ vs $x = 5$ subobjetivos de $d' = 6$:

Sin subobjetivos: 5^{30} . Aplicando $\log_{10}$:

$$\log_{10}(5^{30}) = 30 \cdot \log_{10}(5) \approx 30 \times 0.699 = 20.97$$

Entonces $5^{30} \approx 10^{20.97} \approx 9.3 \times 10^{20}$.

Con subobjetivos: $6 \cdot 5^6 = 6 \times 15.625 = 93.750 \approx 9.4 \times 10^4$.

Ratio:

$$\frac{9.3 \times 10^{20}}{9.4 \times 10^4} \approx 10^{16}$$

Diez mil billones de veces más rápido.

Logaritmos útiles para memorizar:

n	2	3	5	7	10
$\log_{10}(n)$	0.301	0.477	0.699	0.845	1

3.4 Completitud y optimalidad de subobjetivos

Propiedad	Se cumple si...
Completa	Método base completo Y los subobjetivos están en algún camino válido inicio→meta

Óptima	Método base óptimo Y los subobjetivos están en el camino óptimo real
--------	--

Riesgo crítico a conocer

Si los subobjetivos están mal elegidos:

- **Pérdida de optimalidad:**

encontrás solución pero no la mejor.

- **Pérdida de completitud:**

no encontrás solución aunque exista en el problema original.

El cuello de botella de la técnica es la calidad de los subobjetivos. Elegirlos bien requiere conocimiento del dominio (heurística fuerte).

4. Búsqueda online

4.1 La motivación

En algunos entornos no es posible o útil calcular un plan completo antes de actuar:

- **Entornos reactivos:** hay que decidir en milisegundos (videojuegos).
- **Entornos no deterministas:** las acciones tienen resultados inciertos.
- **Entornos dinámicos:** el mundo cambia mientras pensás (escalera de Hogwarts).
- **Entornos inaccesibles:** información parcial, se descubre al actuar (laberinto).

Búsqueda online	Búsqueda offline
1. Percibir entorno 2. Elegir mejor acción 3. Ejecutar acción (repetir)	1. Percibir entorno 2. Buscar plan completo 3. Ejecutar plan

4.2 Ascenso de colinas (hill climbing)

Idea central

En cada paso, mirar los vecinos inmediatos y elegir el que tenga menor h (más cerca de la meta). Ejecutar esa acción. Repetir.

Repetir:

```
nodo ← percibir(entorno)
Si meta?(nodo) → devolver éxito
sucesores ← expandir(nodo)
Si vacía?(sucesores) → devolver fallo
mejor ← argmin[ evaluar(n, h) ] para n en sucesores
a ← acción(nodo, mejor)
ejecutar(a, entorno)
```

Fin repetir

Analogía: caminante en la niebla

Estás en una montaña con niebla. No ves la cima, pero sentís con los pies hacia dónde es más arriba. En cada paso, elegís el más ascendente. Eventualmente llegás a una cima — pero no necesariamente la más alta.

4.3 El problema de los óptimos locales

Duda de la sesión: ¿qué pasa cuando los vecinos empeoran?

Ascenso de colinas es un algoritmo *voraz* (greedy). Al evaluar los vecinos, si todos tienen heurística peor que el estado actual, el algoritmo concluye que "llegó a la mejor solución posible" y se queda atrapado. Este es el problema de **óptimos locales**: cimas secundarias que no son la meta real pero parecen serla desde la vecindad.

Mitigaciones: reinicios aleatorios, búsqueda local iterada, búsqueda tabú (Battiti & Tecchiolli, 1994), optimización reactiva.

4.4 Búsqueda con horizonte

Idea central

Expandir un árbol hasta profundidad k (horizonte) desde el estado actual, evaluar las hojas con h , ejecutar la primera acción que lleva a la mejor hoja. Repetir.

Repetir:

```
nodo ← percibir(entorno)
Si meta?(nodo) → devolver éxito
Expandir árbol de profundidad k desde nodo
H ← hojas a profundidad k
mejor_hoja ← argmin[ h(hoja) ] para hoja en H
primera_acción ← primera acción del camino a mejor_hoja
ejecutar(primera_acción)
```

Fin repetir

Relación con ascenso de colinas

Ascenso de colinas es búsqueda con horizonte $k=1$. Aumentar k permite evitar bucles de longitud menor que k .

k	Comportamiento	Ciclos evitados	Coste por paso
1	= Hill climbing	Ninguno	$O(b)$
3	Visión 3 pasos	Longitud < 3	$O(b^3)$
∞	= A* con lookahead total	Todos	$O(b^d)$

Trade-off reactividad vs deliberación

k pequeño = rápido pero atrapable en óptimos locales. k grande = robusto pero caro. Se ajusta según restricciones temporales del problema. En un juego a 60 FPS (16 ms/frame) probablemente $k = 1$ o $k = 2$. En un planificador logístico con 2 minutos podés usar $k = 20$ o más.

4.5 Optimización mediante búsqueda online

Las búsquedas online son especialmente útiles para **problemas de optimización**, donde:

- Hay una función objetivo a maximizar/minimizar.
- El camino no importa, solo el estado final.
- Basta con un valor "suficientemente bueno", no el óptimo exacto.

Ejemplo: n-reinas

Colocar n reinas en un tablero $n \times n$ sin que se ataquen. No importa cómo llegaste a la configuración final, solo que sea válida. Ideal para ascenso de colinas: empezás con una configuración cualquiera y en cada paso movés una reina para reducir el número de ataques.

Contraste con problemas de ruta: si buscás cómo ir de Arad a Bucarest, el camino importa (necesitás la secuencia). A* es ideal. En n-reinas, A* sería un desperdicio.

4.6 Propiedades de las búsquedas online

Propiedad	Ascenso colinas	Horizonte k
Compleitud	No (general)	No (general)
Optimalidad	No (general)	No (general)
	Sí a ambas	Sí a ambas

Con $h = h^*$		
Memoria	Mínima	$O(b^k)$
Velocidad por paso	Muy rápida	Ajustable con k

5. Tabla comparativa final del Tema 5

Algoritmo	Heurística	Completo	Óptimo	Memoria	Cuándo usarlo
A*	Débil	Sí	Sí si h admisible	Alta	Plan completo, entorno conocido, offline
Subobjetivos	Fuerte	Condicional	Condicional	Media	Problemas profundos con puntos de paso naturales
Ascenso colinas	Fuerte	No	No	Mínima	Reactividad máxima, optimización
Horizonte k	Fuerte	No	No	$O(b^k)$	Balance reactividad-deliberación

6. Test de autoevaluación (claves)

Preguntas y respuestas comentadas

1. En A* se usan heurísticas para:

B. Guiar la búsqueda. (Aplicación débil: ordenan prioridades sin recortar ramas.)

2. La aplicación fuerte de heurísticas hace que:

A + C. Reduce el espacio de búsqueda Y busca soluciones buenas aunque no óptimas. (B es falso: no garantiza la mejor.)

3. El objetivo de la búsqueda por subobjetivos es:

B + C. Reducir la complejidad Y subdividir el problema en subproblemas más pequeños mediante heurística fuerte.

4. Entornos donde no es posible/útil un plan completo:

A + B + C. No deterministas, inaccesibles, y agentes reactivos — los tres escenarios.

5. La idea básica de búsqueda online:

A. Combinar un paso de búsqueda con un paso de actuación.

6. Idea subyacente en ascenso de colinas:

A + B. Reducir profundidad del árbol y siempre ejecutar la acción más prometedora, con ciclo percepción-acción continuado.

7. Completitud en ascenso de colinas se consigue si:

B + C. Conjunto de estados finito Y eliminar estados repetidos (previene bucles infinitos).

8. En búsqueda con horizonte:

A. La primera acción de un plan que lleva a una hoja con evaluación óptima tiene alta probabilidad de pertenecer al camino óptimo. (C es falso: no garantiza optimalidad.)

9. En un problema de optimización:

B + C. No se busca un estado exacto sino uno que se acerque al objetivo, evaluando según función objetivo. (A es falso: el camino suele ser irrelevante.)

10. La búsqueda online para optimización es útil para:

C. Tanto número infinito como finito de estados. La técnica no depende de la cardinalidad del espacio.

7. Dudas y claves de la sesión

Confusiones que surgieron y cómo se resolvieron

Confusión FIFO/LILO:

BFS usa FIFO (First-In-First-Out, cola) y DFS usa LIFO (Last-In-First-Out, pila). Evitar el error "LILO" que no existe como acrónimo estándar.

Confusión nodo vs pieza en puzzle-8:

un nodo es el tablero completo 3×3 (configuración), no una ficha individual. Las transiciones entre nodos son movimientos del espacio vacío.

Confusión admisibilidad vs consistencia:

admisibilidad es global ($h(n) \leq h^*(n)$ para cualquier nodo a meta). Consistencia es

local (coherencia entre vecinos, $h(s) \leq c(s, s') + h(s')$). Toda consistente es

admisible, pero no al revés.

Confusión "monotonía creciente":

NO significa que f tenga que subir estrictamente en cada paso — significa que

nunca decrece. Puede mantenerse igual o subir, pero jamás bajar.

Confusión admisibilidad → optimalidad:

con heurística inadmisibile se pierde optimalidad pero NO completitud. A* sigue encontrando alguna solución, solo que no garantiza la mejor.

Claves para el examen

- A* termina cuando extrae la meta, NO cuando la agrega a la cola.
- $h(n) = 0 \rightarrow A^*$ se convierte en UCS.
- Aplicación débil (A^*) = guía; aplicación fuerte (subobjetivos, hill climbing) = recorte.
- Ascenso de colinas = búsqueda con horizonte con $k = 1$.
- Subobjetivos: completitud y optimalidad son CONDICIONALES a cómo están elegidos.
- En problemas de optimización, el camino no importa — solo el estado final.
- Para estimar magnitud de b^d : usar $\log_{10}(b^d) = d \cdot \log_{10}(b)$.